

Autonomous Go-Kart Navigation Using LIDAR-Based Obstacle Tracking and Pure Pursuit Path Planning

Seth Jones, Dante Palmieri, Evan Hudak, Devon Allan
Dr. Rodolfo Echavarria Solis | EE476-C

Abstract—This paper serves to present the current progress of design, testing, and deployment of a Go-Kart that is capable of autonomously navigating a given environment based on context clues. It will also be able to identify, manage, and respond to both static and dynamic obstacles using a Light Detection and Ranging (LIDAR) based perception pipeline with support of an Inertial Measurement Unit (IMU) [10,12].

The electrical engineering team is responsible for the sensors, computation, control subsystems, and power supply; while the mechanical engineering team is responsible for implementing mechanical interfaces from chosen motors to the cart in the steering, braking, and throttle assemblies [15]. The system plans to use two 360 degree LIDAR units for environmental sensing and range finding [10]; these are supported by three separate IMU feedback points for obstacle tracking [12]. Point-cloud topology gradients enable “naming” obstacles, allowing persistent identification over time [4].

A pure-pursuit controller computes the optimal trajectory through static obstacles, and an arbitration algorithm determines appropriate behaviors in the presence of static or dynamic obstacles in the path given by pure pursuit [5,13,14]. Steering and braking are actuated via high-torque servos, throttle via a stepper motor, and each subsystem is governed by its own Arduino Nano [9]. A Raspberry Pi 5 executes navigation and obstacle-tracking algorithms [8]. This project demonstrates a modular, scalable framework for autonomous navigation using inexpensive sensors and embedded hardware [5,10].

I. Objective

The objective of this project is to develop a fully autonomous Go-Kart capable of reliably navigating a randomly assigned course using pure pursuit [13,14], while also detecting, classifying, and tracking obstacles through 360° LIDAR data [10]. The system must distinguish between static and dynamic obstacles by applying temporal “naming” techniques and IMU-based motion compensation [12], enabling it to execute safe and appropriate maneuvers around or toward obstacles depending on their behavior. To support this functionality, the Go-Kart integrates three independent electromechanical subsystems—steering, braking, and throttle—each controlled by dedicated microcontrollers [9,15]. These components are unified through a robust arbitration framework that merges the outputs of both the navigation and obstacle-tracking modules, ensuring cohesive and reliable autonomous operation [5].

II. Introduction

Autonomous vehicle navigation remains a central challenge in robotics, transportation, and embedded systems research [5]. Many advanced platforms rely on costly sensor suites such as multi-beam LIDAR, high-resolution cameras, and industrial-grade IMUs [10,12]. This project provides a low-cost alternative using a Go-Kart platform equipped with hobby-grade components while still demonstrating advanced navigation behaviors [8,9,10].

Earlier iterations of the project used stereo cameras and reinforcement learning to navigate static courses. However, due to changes in hardware availability, client feedback, and the desire for higher reliability and interpretability, the new system transitioned to a LIDAR-only perception pipeline supported by classical control methods [10,12,13].

This paper presents the complete hardware and software architecture, including:

- LIDAR-based obstacle detection with topological gradient analysis,
- Obstacle naming and time-series tracking,
- Static vs. dynamic obstacle classification,
- Pure-pursuit navigation,
- Actuation using high-torque servo motors and stepper motors,
- Distributed microcontroller architecture for reliability.

III. Materials

A. Mechanical Systems

The mechanical engineering team contributed three independent subsystems:

A Steering Assembly

Modified steering column driven by high-torque servo.

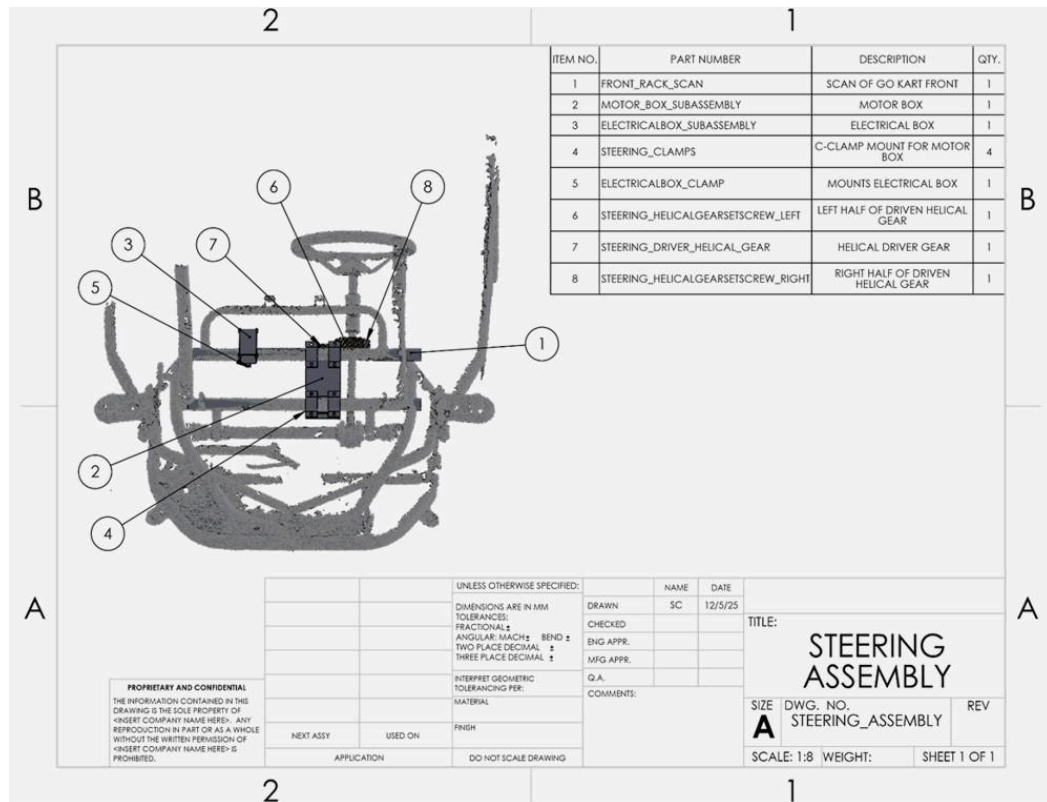


Fig. 1. Steering Full CAD Assembly

B Braking Assembly

Mechanical brake lever connected to high-torque servo.

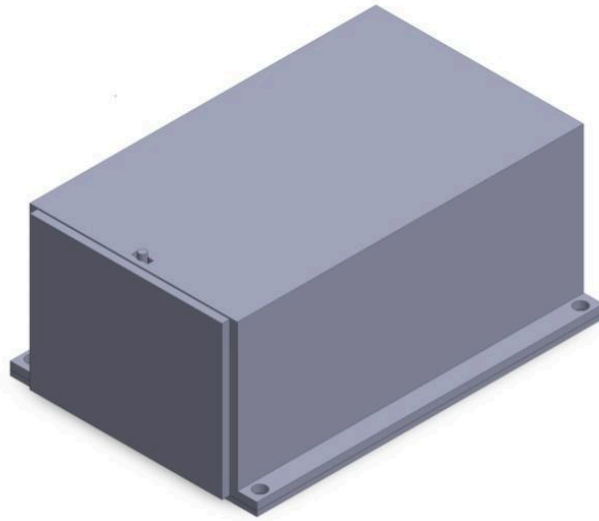


Fig. 2. Braking Motor Mount CAD Assembly

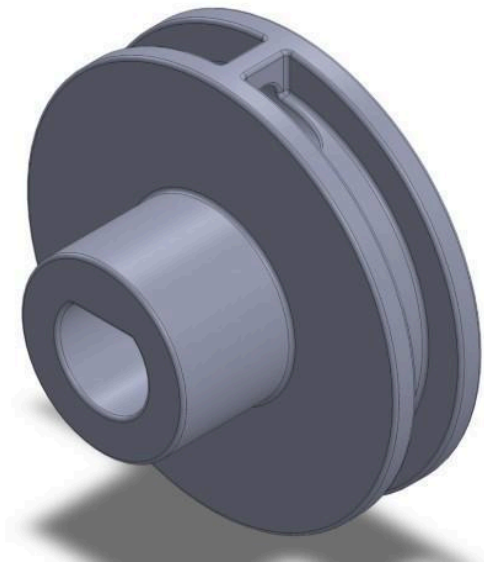


Fig. 3. Braking Spool CAD Assembly

C Throttle Assembly

Throttle lever controlled via stepper motor and encoder to track pedal position.

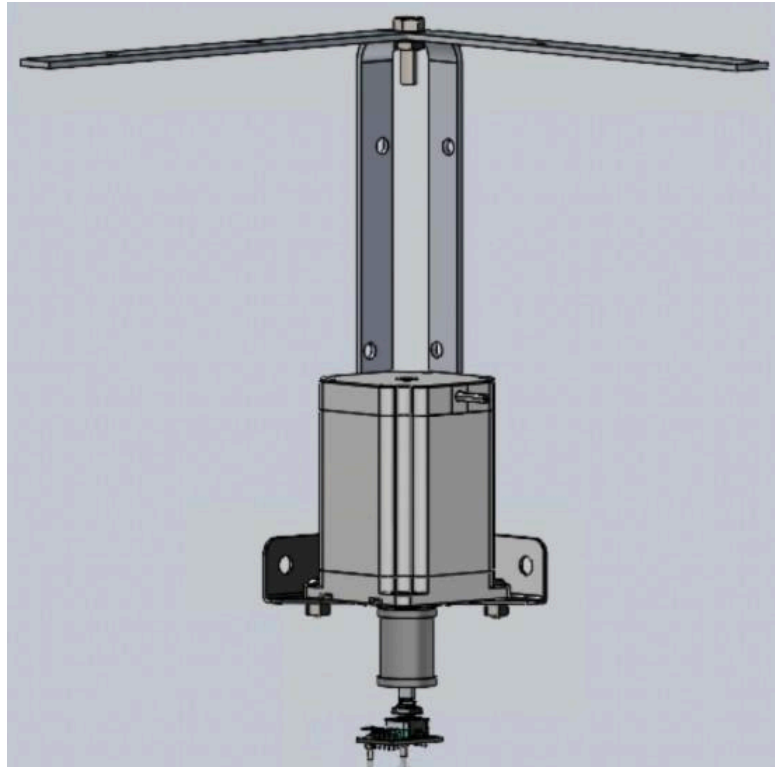


Fig. 4. Throttle Motor Mount CAD Assembly

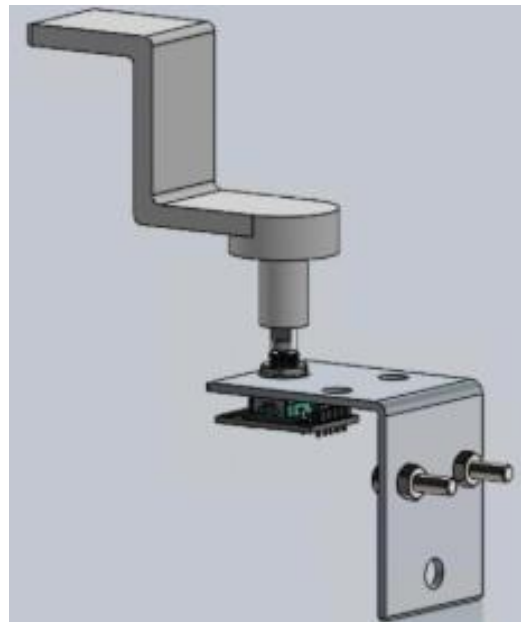


Fig. 5. Throttle Pedal Mount CAD Assembly

B. Electrical and Computing Hardware

Component	Quantity	Purpose
Raspberry Pi 5 (8 GB)	1	Runs navigation, obstacle detection, arbitration
Arduino Nano	3	Independent steering, throttle, and braking controllers
360° LIDAR Units	2	Full-surround environmental sensing
IMU (3-axis)	3	Motion compensation for obstacle tracking
High-torque servo motors	2	Steering and braking actuation
Stepper motor	1	Throttle actuation
Stepper Driver	1	Stepper Control
24V 60Ah LiPo battery	1	High-current motor power
USB power bank	1	Low-voltage power for Pi, LIDAR, and Arduinos
Encoder	3	For determining steppes and pedal positions

C. Software Tools

1. **ROS2** for middleware and modular communication
2. **Python** / **C#** for algorithm development
3. **Open3D** for point-cloud processing
4. **NumPy** / **C Math Library** for obstacle tracking computation
5. **Fusion 360** for electronics and mounting design
6. **Unity** for simulation testing

IV. Standards

The system complies with the following engineering standards:

1. **Power and Wiring Standards**
 - IEEE Std 315-1975 for schematic representation
 - NEC low-voltage wiring guidelines
2. **Communication and Data Standards**
 - ROS2 DDS communication protocol
 - I²C, SPI, and UART standards
3. **Safety Standards**
 - ISO 26262-inspired fail-safe requirements
 - Manual mechanical override system
4. **LIDAR Data Processing Standards**
 - PCL-based 3D point-cloud operations

V. Results

The autonomous Go-Kart project has reached the initial hardware integration phase, and preliminary results focus on system readiness, sensor installation, and software pipeline validation. At this stage, no full-scale mechanical or navigation testing has been conducted.

A. Sensor Integration

Two 360° LIDAR units have been received and mounted on the Go-Kart chassis. Mechanical engineering has constructed the mounting brackets, ensuring proper alignment and clearance. LIDAR units are powered via a dedicated phone battery pack, independent of the main 24V motor battery, reducing electrical interference. Initial power-on tests confirm connectivity and data streaming to the Raspberry Pi 5.

There will be a stepper motor encoder for throttle, and brake and throttle pedal encoders, are connected and verified for signal transmission. A steering potentiometer is installed for feedback to the steering controller.

The IMU will be mounted and successfully stream acceleration and angular velocity data to the perception pipeline.

B. Processing and Control Pipelines

The software pipelines, as represented in the flowchart, have been partially validated in simulation and with offline data:

A Perception Pipeline

LIDAR point cloud data can be received and pre-processed on the Raspberry Pi. Segmentation, clustering, and obstacle naming routines are functional in simulation; real-time performance with live LIDAR data is currently being assessed.

B Navigation Pipeline

Pure-pursuit path calculation has been verified using predefined waypoint datasets. No real-world path-following tests have been conducted due to pending actuator testing.

C Arbitration Pipeline

Logic for determining dynamic versus static obstacles and deciding stop or reroute actions has been implemented in software. Currently validated against synthetic obstacle datasets.

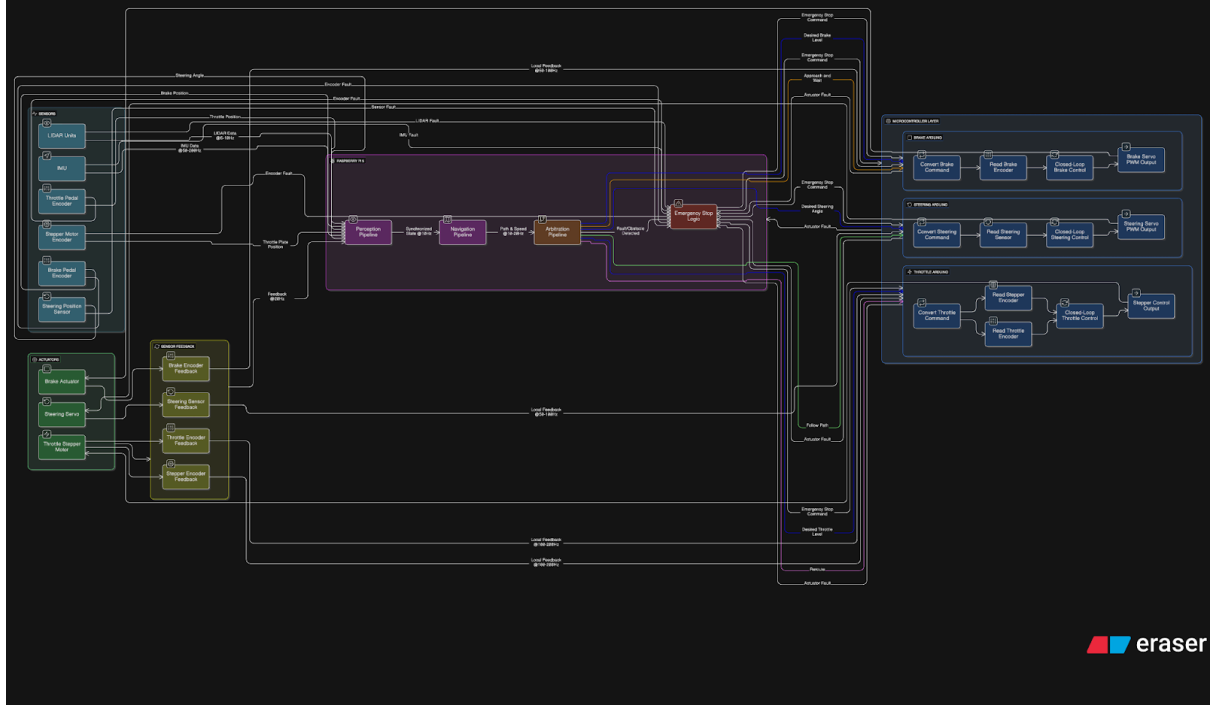


Fig. 6. Data Flow Chart

C. Microcontroller and Actuator Readiness

Three dedicated Arduinos have been configured for steering, braking, and throttle control. Each Arduino can accept commands from the Raspberry Pi and correctly process encoder feedback. No mechanical testing of servos or stepper motors under load has been performed.

D. System Constraints and Safety Checks

Fail-safe and emergency stop routines are implemented in software. Health checks for LIDAR connectivity, encoder feedback, and communication with Arduinos have been tested in simulation and with individual components. Mechanical testing of the braking and steering systems is pending; therefore, full end-to-end testing of emergency stop behavior has not yet been performed.

E. Summary

At this stage, the project demonstrates:

1. Successful hardware integration of sensors and encoders.

2. Functional software pipelines capable of perception, navigation, and arbitration in simulation or offline data.
3. Controller and actuator interfaces configured and receiving commands.

F. Next Steps

1. Mechanical engineers will perform structural testing of mounts and actuators.
2. Live testing of LIDAR perception and obstacle tracking.
3. Integration of perception, navigation, and arbitration pipelines with actuators for initial closed-loop testing.

VI. Conclusions

The electrical engineering team has successfully established the foundational hardware and software architecture necessary to meet the requirements for an autonomous Go-Kart that is capable of navigating complex environments without any direct user input. At this stage, the team has:

A. Integrated key hardware components

Including dual 360° LIDAR units, IMUs, high-torque servo motors, a stepper motor for throttle, and dedicated Arduinos for each actuation subsystem. Mechanical mounts for all actuators and sensors have been constructed, ensuring proper alignment and readiness for testing.

B. Implemented and partially validated software pipelines

Including perception, environment fabrication, navigation, obstacle arbitration, and dynamic obstacle handling modules. The perception and environment fabrication pipeline will be capable of obstacle segmentation, “naming”, and temporal tracking in simulation, while the navigation and arbitration logic will compute paths and determine obstacle responses in data.

C. Established safety and control mechanisms

This includes software fail-safes, health monitoring of sensors and actuators, and emergency stop routines for manual over-ride mode.

D. Current State

Although no full scale testing has been performed yet, the project architecture shows promise of a modular, yet scalable framework that separates computation, navigation, and actuation; facilitating future development and testing for the coming semester.

E. Next steps

The next steps for this project include mechanical testing of mounts and power transmission, moving LIDAR from simulation into the real world, and closed-loop integration of environment perception, advanced navigation, arbitration units, and electro-mechanical actuators. Once completed, the Go-Kart will have the ability to autonomously navigate complex and changing

courses while also responding safely to obstacles in its path, thus fulfilling the original design objectives.

VII. Acknowledgements

The team would like to express their sincere gratitude to the following individuals and organizations for their support and guidance throughout this project:

- **Dr. Rodolfo Echavarria Solis**, for providing mentorship, technical guidance, and feedback on both hardware and software design aspects of the project through presentations.
- **The Mechanical Engineering Team**, for designing and constructing the actuator mounts and mechanical interfaces for the steering, braking, and throttle assemblies.
- **Michael Logan Garret**, for providing the resources, time, and technical support necessary to integrate sensors and navigation algorithms.
- **Open-Source Software Communities**, including ROS 2, Open3D, and Unity, for providing tools and frameworks critical to simulation, perception, and control algorithm development.

This project would not have been possible without the collaborative efforts and support of all the individuals and teams listed above.

VIII. Appendix

A. CAD Files

Fig 1 -

<https://drive.google.com/file/d/1f0ndeVKpNL7V8HgL6nfsesRY8De-bL9f/view?usp=sharing>

Fig 2-3-

<https://drive.google.com/file/d/1OtG5U2xMRLAVoaZdEcZmBU9BVFJGqP9D/view?usp=sharing>

Fig 4 -

<https://drive.google.com/file/d/19msAX6lTdfye9xrzCimIE0qtmVC3kbT0/view?usp=sharing>

Fig 5 -

<https://drive.google.com/file/d/1sSTvtxhgpkcMK7Hf-jMPxzRtXgnLNvAD/view?usp=sharing>

B. Data Sheets

LIDAR -

<https://drive.google.com/file/d/1DhHre15Nqku7ccdebwcsTU5Zgz7MwU8U/view?usp=sharing>

IMU -

[GY-521.pdf](#)

Arduino Nano -

[Nano | Arduino Documentation](#)

Raspberry PI 5 -

[Raspberry Pi 5 Single Board Computer SBC v2 Data-3317538.pdf](#)

C. System Diagram

Fig 6 -

<https://drive.google.com/file/d/1BgltX4xgl9VOX3xOvWsHiTOTMWCZiVA4/view?usp=sharing>

D. Listings

24V Battery -

[Amazon.com: KEPWORTH 24V Lithium Battery 60Ah LiFePO4 Batteries, Built-In 50A BMS with Low-Temp Protection, 4000+ Deep Cycle, for Boat, Solar System : Automotive](#)

References

- [1] IEEE Std 315-1975, *Graphic Symbols for Electrical and Electronics Diagrams*, IEEE, 1975.
- [2] National Electrical Code (NEC), *Low-Voltage Wiring Guidelines*, National Fire Protection Association (NFPA).
- [3] ISO 26262, *Road Vehicles – Functional Safety*, International Organization for Standardization, 2011.

- [4] Open3D: *Open-Source Library for 3D Data Processing*, <http://www.open3d.org>, accessed Dec. 2025.
- [5] Robot Operating System 2 (ROS2), *Middleware for Robot Software Development*, <https://index.ros.org/doc/ros2/>, accessed Dec. 2025.
- [6] Fusion 360, *Autodesk CAD and Electronics Design Software*, Autodesk Inc., <https://www.autodesk.com/products/fusion-360>, accessed Dec. 2025.
- [7] Unity, *Simulation and Visualization Platform*, Unity Technologies, <https://unity.com/>, accessed Dec. 2025.
- [8] Raspberry Pi Foundation, *Raspberry Pi 5 Single Board Computer Datasheet*, 2023.
- [9] Arduino, *Arduino Nano Documentation*, <https://docs.arduino.cc/hardware/nano>, accessed Dec. 2025.
- [10] Slamtec, *RPLIDAR A1/A2 Datasheet*, Slamtec Inc., <https://www.slamtec.com/en/Lidar/A1>, accessed Dec. 2025.
- [11] KEPWORTH, *24V 60Ah LiFePO4 Battery Datasheet*, Amazon.com.
- [12] InvenSense, *GY-521 3-Axis IMU Datasheet*, <https://www.invensense.com/products/motion-tracking/9-axis/mpu-9250/>, accessed Dec. 2025.
- [13] NumPy: *Fundamental Package for Scientific Computing with Python*, <https://numpy.org>, accessed Dec. 2025.
- [14] C Math Library, *Standard C Mathematical Functions*, ISO/IEC 9899:2018.
- [15] Go-Kart CAD Files (Steering, Braking, Throttle):
- Fig 1:
<https://drive.google.com/file/d/1f0ndeVKpNL7V8HgL6nfsesRY8De-bL9f/view?usp=sharing>
 - Fig 2–3:
<https://drive.google.com/file/d/1OtG5U2xMRLAVoaZdEcZmBU9BVFJGqP9D/view?usp=sharing>
 - Fig 4:
<https://drive.google.com/file/d/19msAX6lTdfye9xrzCimIE0qtmVC3kbT0/view?usp=sharing>
 - Fig 5:
<https://drive.google.com/file/d/1sSTvtxhgpkcMK7Hf-jMPxzRtXgnLNvAD/view?usp=sharing>
- [16] System Diagram: Fig 6:
<https://drive.google.com/file/d/1BgltX4xgI9VOX3xOvWsHiTOTMWCZiVA4/view?usp=sharing>
- [17] LIDAR Datasheet:
<https://drive.google.com/file/d/1DhHre15Nqku7ccdebwcsTU5Zgz7MwU8U/view?usp=sharing>

